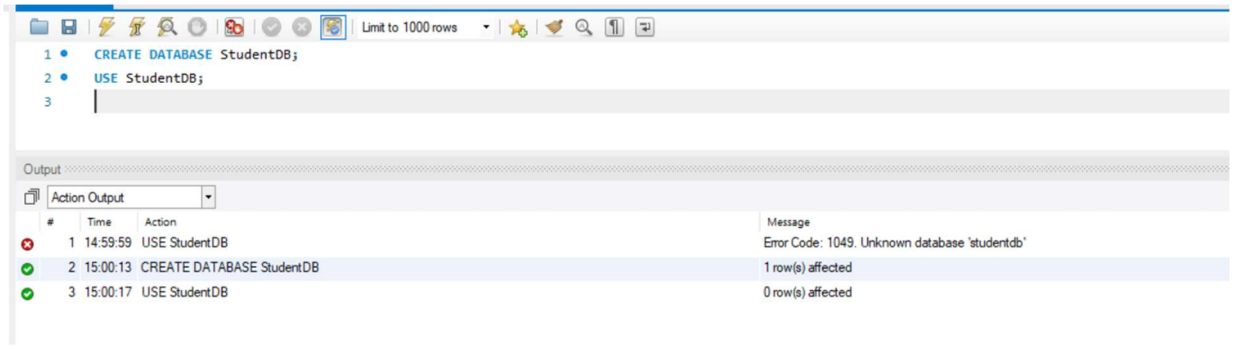
Create a Student Database



```
CREATE DATABASE StudentDB;
USE StudentDB;
```

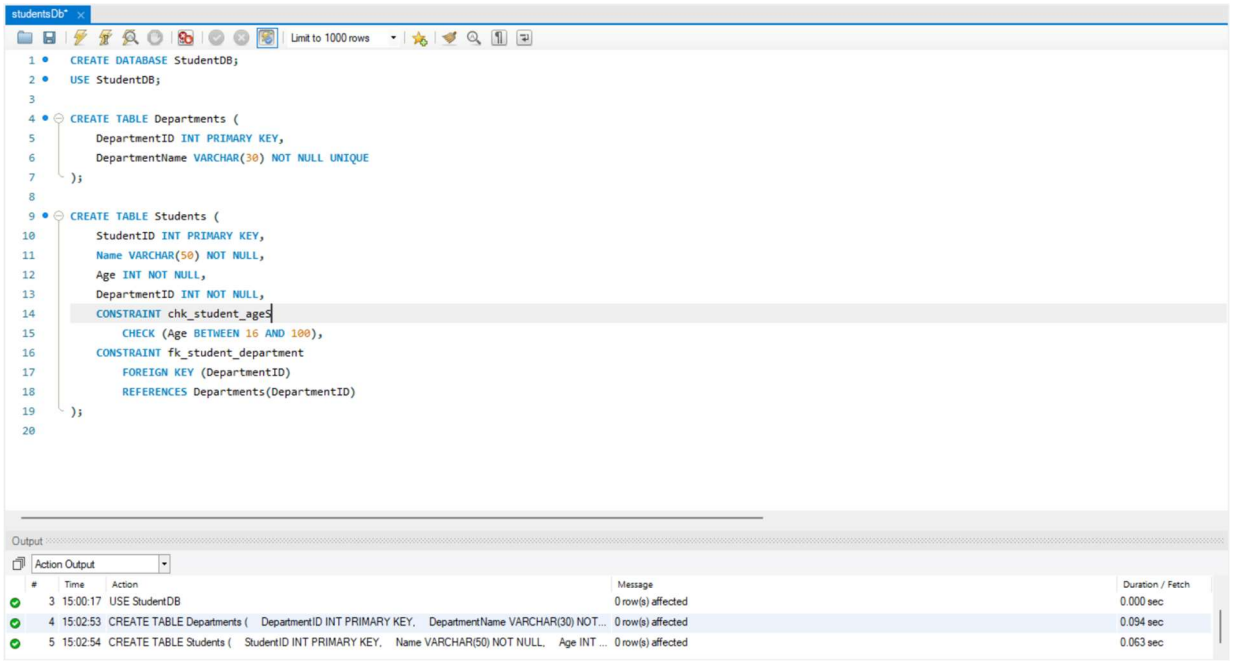
Output

Action Output

#	Time	Action	Message
1	14:59:59	USE StudentDB	Error Code: 1049. Unknown database 'studentdb'
2	15:00:13	CREATE DATABASE StudentDB	1 row(s) affected
3	15:00:17	USE StudentDB	0 row(s) affected

Create the tables with constraints

Students Table and Departments Table Creation :



```
CREATE DATABASE StudentDB;
USE StudentDB;

CREATE TABLE Departments (
    DepartmentID INT PRIMARY KEY,
    DepartmentName VARCHAR(30) NOT NULL UNIQUE
);

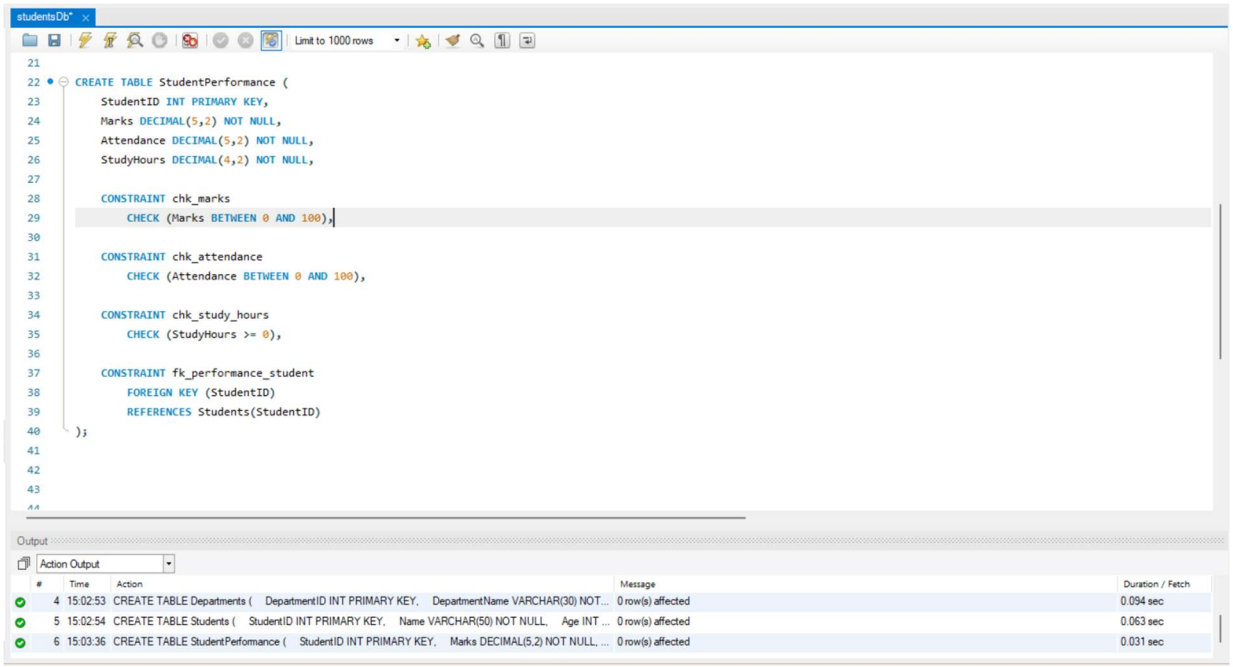
CREATE TABLE Students (
    StudentID INT PRIMARY KEY,
    Name VARCHAR(50) NOT NULL,
    Age INT NOT NULL,
    DepartmentID INT NOT NULL,
    CONSTRAINT chk_student_ageS
        CHECK (Age BETWEEN 16 AND 100),
    CONSTRAINT fk_student_department
        FOREIGN KEY (DepartmentID)
        REFERENCES Departments(DepartmentID)
);
```

Output

Action Output

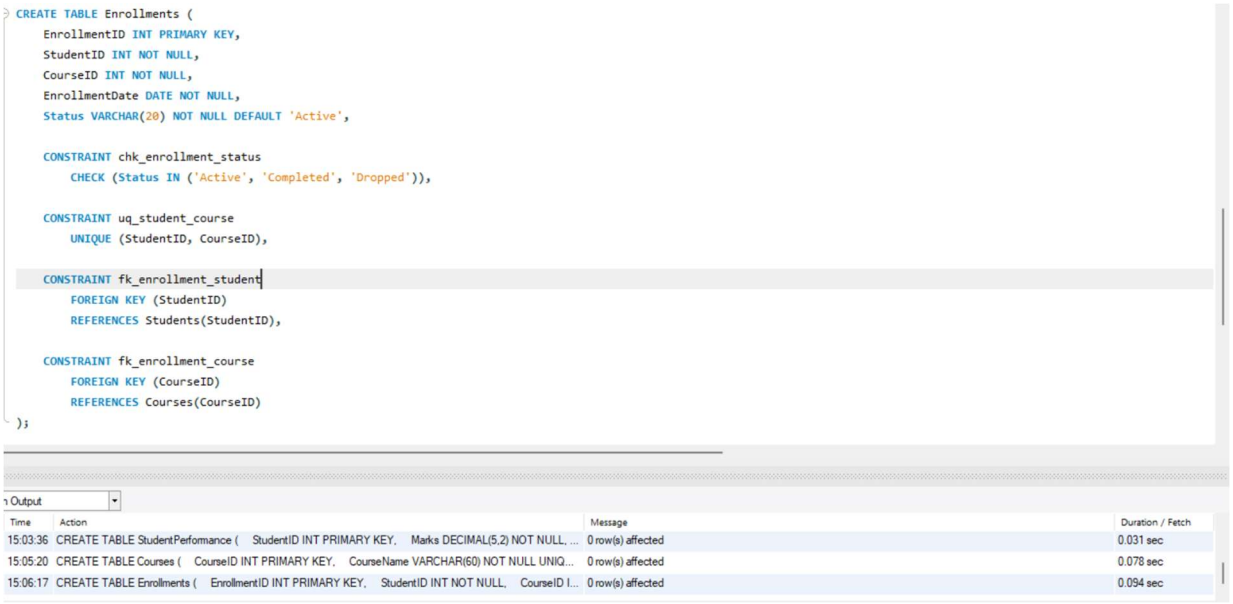
#	Time	Action	Message	Duration / Fetch
3	15:00:17	USE StudentDB	0 row(s) affected	0.000 sec
4	15:02:53	CREATE TABLE Departments (DepartmentID INT PRIMARY KEY, DepartmentName VARCHAR(30) NOT...	0 row(s) affected	0.094 sec
5	15:02:54	CREATE TABLE Students (StudentID INT PRIMARY KEY, Name VARCHAR(50) NOT NULL, Age INT ...	0 row(s) affected	0.063 sec

Creating Student Performance Table



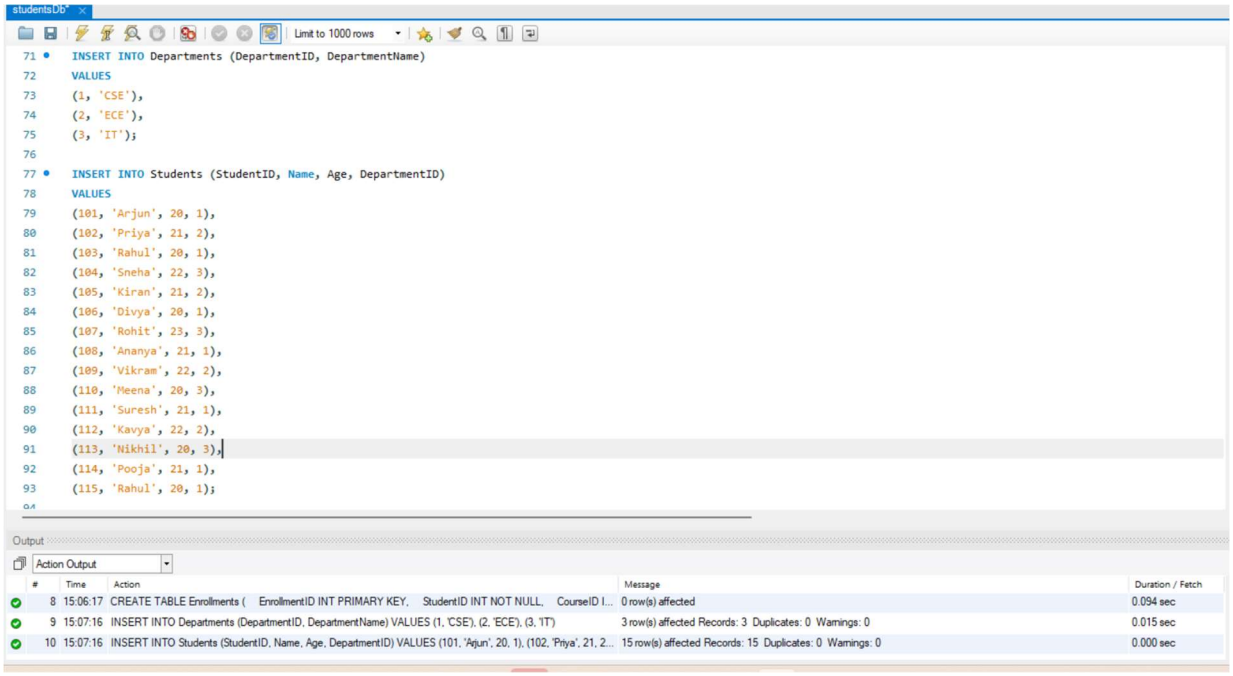
Courses table:





Inserting Data :

Departments and Students



```
INSERT INTO Departments (DepartmentID, DepartmentName)
VALUES
(1, 'CSE'),
(2, 'ECE'),
(3, 'IT');

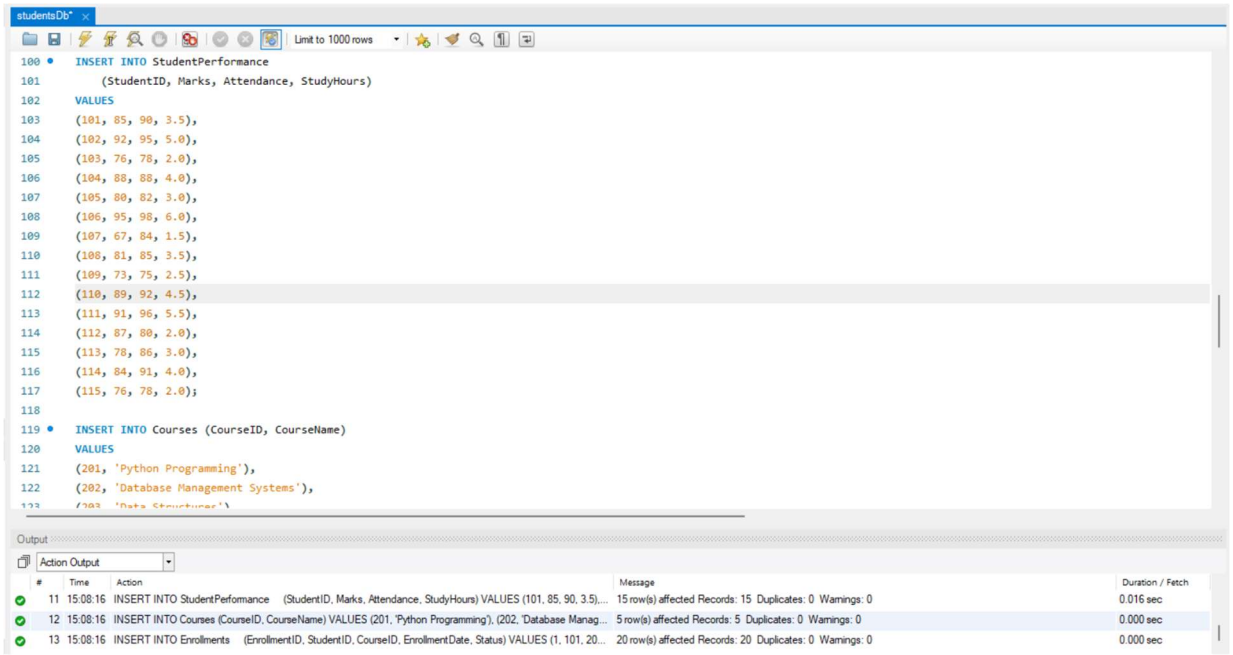
INSERT INTO Students (StudentID, Name, Age, DepartmentID)
VALUES
(101, 'Arjun', 20, 1),
(102, 'Priya', 21, 2),
(103, 'Rahul', 20, 1),
(104, 'Sneha', 22, 3),
(105, 'Kiran', 21, 2),
(106, 'Divya', 20, 1),
(107, 'Rohit', 23, 3),
(108, 'Ananya', 21, 1),
(109, 'Vikram', 22, 2),
(110, 'Meena', 20, 3),
(111, 'Suresh', 21, 1),
(112, 'Kavya', 22, 2),
(113, 'Nikhil', 20, 3),
(114, 'Pooja', 21, 1),
(115, 'Rahul', 20, 1);
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
8	15:06:17	CREATE TABLE Enrollments (EnrollmentID INT PRIMARY KEY, StudentID INT NOT NULL, CourseID I...	0 row(s) affected	0.094 sec
9	15:07:16	INSERT INTO Departments (DepartmentID, DepartmentName) VALUES (1, 'CSE'), (2, 'ECE'), (3, 'IT')	3 row(s) affected Records: 3 Duplicates: 0 Warnings: 0	0.015 sec
10	15:07:16	INSERT INTO Students (StudentID, Name, Age, DepartmentID) VALUES (101, 'Arjun', 20, 1), (102, 'Priya', 21, 2...	15 row(s) affected Records: 15 Duplicates: 0 Warnings: 0	0.000 sec

Student Performance and Courses and Enrollments



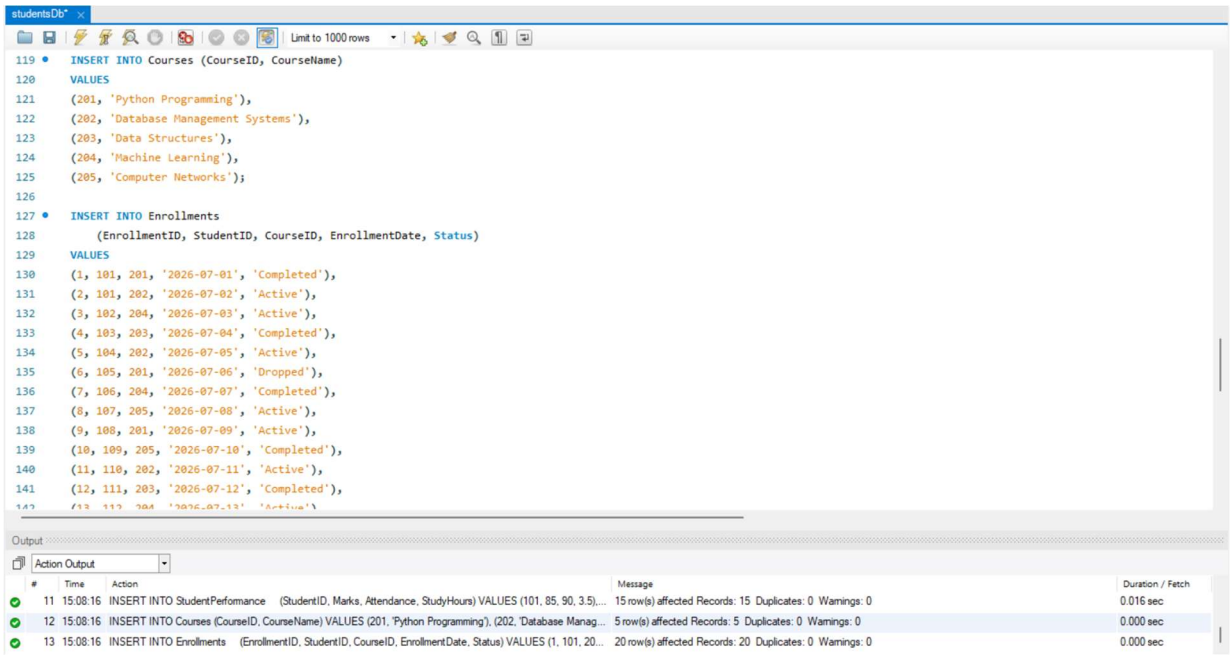
```
INSERT INTO StudentPerformance
    (StudentID, Marks, Attendance, StudyHours)
VALUES
(101, 85, 90, 3.5),
(102, 92, 95, 5.0),
(103, 76, 78, 2.0),
(104, 88, 88, 4.0),
(105, 80, 82, 3.0),
(106, 95, 98, 6.0),
(107, 67, 84, 1.5),
(108, 81, 85, 3.5),
(109, 73, 75, 2.5),
(110, 89, 92, 4.5),
(111, 91, 96, 5.5),
(112, 87, 80, 2.0),
(113, 78, 86, 3.0),
(114, 84, 91, 4.0),
(115, 76, 78, 2.0);

INSERT INTO Courses (CourseID, CourseName)
VALUES
(201, 'Python Programming'),
(202, 'Database Management Systems'),
```

Output

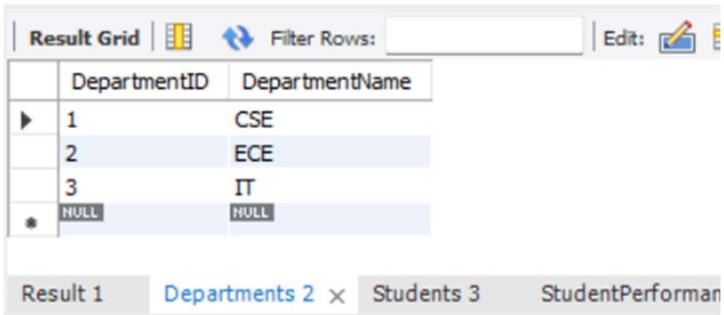
Action Output

#	Time	Action	Message	Duration / Fetch
11	15:08:16	INSERT INTO StudentPerformance (StudentID, Marks, Attendance, StudyHours) VALUES (101, 85, 90, 3.5),...	15 row(s) affected Records: 15 Duplicates: 0 Warnings: 0	0.016 sec
12	15:08:16	INSERT INTO Courses (CourseID, CourseName) VALUES (201, 'Python Programming'), (202, 'Database Manag...	5 row(s) affected Records: 5 Duplicates: 0 Warnings: 0	0.000 sec
13	15:08:16	INSERT INTO Enrollments (EnrollmentID, StudentID, CourseID, EnrollmentDate, Status) VALUES (1, 101, 20...	20 row(s) affected Records: 20 Duplicates: 0 Warnings: 0	0.000 sec



Checking the inserted data





Result Grid | Filter Rows: | Edit:

StudentID	Name	Age	DepartmentID
101	Arjun	20	1
102	Priya	21	2
103	Rahul	20	1
104	Sneha	22	3
105	Kiran	21	2

Result Grid | Filter Rows:

StudentID	Marks	Attendance	StudyHours
101	85.00	90.00	3.50
102	92.00	95.00	5.00
103	76.00	78.00	2.00
104	88.00	88.00	4.00
105	80.00	82.00	3.00

Result Grid | Filter Rows:

CourseID	CourseName
205	Computer Networks
203	Data Structures
202	Database Management Systems
204	Machine Learning
201	Python Programming

Result Grid | Filter Rows: | Edit: | Ex

EnrollmentID	StudentID	CourseID	EnrollmentDate	Status
1	101	201	2026-07-01	Completed
2	101	202	2026-07-02	Active
3	102	204	2026-07-03	Active
4	103	203	2026-07-04	Completed
5	104	202	2026-07-05	Active

Result 1 Departments 2 Students 3 StudentPerformance 4 Co

Checking the Table Columns and datatypes :

```
160 •  DESCRIBE Students;
161
162
163
164
165
166
167
```

Result Grid | Filter Rows: | Export: | Wrap Cell Cont

Field	Type	Null	Key	Default	Extra
StudentID	int	NO	PRI	NULL	
Name	varchar(50)	NO		NULL	
Age	int	NO		NULL	
DepartmentID	int	NO	MUL	NULL	

Queries

1. Find students whose names start with R.

```
162
163 •  SELECT * FROM Students WHERE Name LIKE 'R%';
164
165
166
167
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap

StudentID	Name	Age	DepartmentID
103	Rahul	20	1
107	Rohit	23	3
115	Rahul	20	1
NULL	NULL	NULL	NULL

2. Display the top five students by marks.

```
166 • SELECT StudentID, Marks FROM StudentPerformance ORDER BY Marks DESC LIMIT 5;
167
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content: | Fetch rows:

StudentID	Marks
106	95.00
102	92.00
111	91.00
110	89.00
104	88.00
NULL	NULL

3. Find students from CSE or IT.

```
169 • SELECT * FROM Students WHERE DepartmentID IN (1, 3);
170
171
172
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Conten

StudentID	Name	Age	DepartmentID
101	Arjun	20	1
103	Rahul	20	1
104	Sneha	22	3
106	Divya	20	1
107	Rohit	23	3
108	Ananya	21	1
110	Meena	20	3
111	Suresh	21	1
113	Nikhil	20	3
114	Pooja	21	1
115	Rahul	20	1

Students 10

4. Find students who studied more than three hours, ordered by study time.

```
171 •  SELECT StudentID, StudyHours
172    FROM StudentPerformance
173    WHERE StudyHours > 3
174    ORDER BY StudyHours DESC;
175
```

Result Grid | Filter Rows: | Edit:

StudentID	StudyHours
106	6.00
111	5.50
102	5.00
110	4.50
104	4.00
114	4.00
101	3.50
108	3.50
NULL	NULL

Using Aggregate functions

5.Count the total number of students.

```
178 •  SELECT COUNT(*) AS TotalStudents FROM Students;
179
180
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

TotalStudents
15

6. Calculate average, highest and lowest marks.

```
181 •  SELECT ROUND(AVG(Marks), 2) AS AverageMarks,
182        MAX(Marks) AS HighestMarks,
183        MIN(Marks) AS LowestMarks,
184        SUM(Marks) AS TotalMarks
185    FROM StudentPerformance;
186
187
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

AverageMarks	HighestMarks	LowestMarks	TotalMarks
82.80	95.00	67.00	1242.00

Using GROUP BY and HAVING

7. Count students in each department.

```
187 •  SELECT
188    d.DepartmentName,
189    COUNT(s.StudentID) AS TotalStudents
190    FROM Departments d
191    JOIN Students s
192    ON d.DepartmentID = s.DepartmentID
193    GROUP BY d.DepartmentID, d.DepartmentName;
194
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

DepartmentName	TotalStudents
CSE	7
ECE	4
IT	4

8. Find departments with at least three students.

```
196 •  SELECT
197    d.DepartmentName,
198    COUNT(s.StudentID) AS TotalStudents
199    FROM Departments d
200    JOIN Students s
201    ON d.DepartmentID = s.DepartmentID
202    GROUP BY d.DepartmentID, d.DepartmentName
203    HAVING COUNT(s.StudentID) >= 3;
204
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

DepartmentName	TotalStudents
CSE	7
ECE	4
IT	4

9. Find departments whose average marks exceed 80.

```
206 •  SELECT
207    d.DepartmentName,
208    ROUND(AVG(p.Marks), 2) AS AverageMarks
209    FROM Departments d
210    JOIN Students s
211    ON d.DepartmentID = s.DepartmentID
212    JOIN StudentPerformance p
213    ON s.StudentID = p.StudentID
214    GROUP BY d.DepartmentID, d.DepartmentName
215    HAVING AVG(p.Marks) > 80;
216
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

DepartmentName	AverageMarks
CSE	84.00
ECE	83.00
IT	80.50

Using JOIN queries

10. INNER JOIN — students with their department names

```
219 •  SELECT
220    s.StudentID,
221    s.Name,
222    d.DepartmentName
223    FROM Students s
224    INNER JOIN Departments d
225    ON s.DepartmentID = d.DepartmentID;
226
```

Result Grid | Filter Rows: | Export: | Wrap Cell Co

StudentID	Name	DepartmentName
101	Arjun	CSE
103	Rahul	CSE
106	Divya	CSE
108	Ananya	CSE
111	Suresh	CSE
114	Pooja	CSE
115	Rahul	CSE
102	Priya	ECE
105	Kiran	ECE
109	Vikram	ECE

11. JOIN three tables — student marks and department

```
221 •  SELECT
222    s.Name,
223    d.DepartmentName,
224    p.Marks,
225    p.Attendance
226    FROM Students s
227    JOIN Departments d
228    ON s.DepartmentID = d.DepartmentID
229    JOIN StudentPerformance p
230    ON s.StudentID = p.StudentID
231    ORDER BY p.Marks DESC;
232
```

Result Grid | Filter Rows: | Export: | Wrap Cel

Name	DepartmentName	Marks	Attendance
Divya	CSE	95.00	98.00
Priya	ECE	92.00	95.00
Suresh	CSE	91.00	96.00
Meena	IT	89.00	92.00
Sneha	IT	88.00	88.00
Kavya	ECE	87.00	80.00
Arjun	CSE	85.00	90.00
Pooja	CSE	84.00	91.00
Ananya	CSE	81.00	85.00
Kiran	ECE	80.00	82.00

92.00

Using Subqueries

12. Students who scored above the overall average

```
234 •   SELECT s.StudentID, s.Name, p.Marks
235     FROM Students s
236     JOIN StudentPerformance p
237     ON s.StudentID = p.StudentID
238     WHERE p.Marks > (
239         SELECT AVG(Marks)
240         FROM StudentPerformance
241     )
242     ORDER BY p.Marks DESC;
243
```

Result Grid | Filter Rows: | Export: | Wrap

StudentID	Name	Marks
106	Divya	95.00
102	Priya	92.00
111	Suresh	91.00
110	Meena	89.00
104	Sneha	88.00
112	Kavya	87.00
101	Arjun	85.00
114	Pooja	84.00

13. Find the student or students with the highest marks

```
245 •  SELECT s.Name, p.Marks
246    FROM Students s
247    JOIN StudentPerformance p
248        ON s.StudentID = p.StudentID
249    WHERE p.Marks = (
250        SELECT MAX(Marks)
251        FROM StudentPerformance
252    );
253
```

Result Grid | Filter Rows: | Export: | W

Name	Marks
Divya	95.00

14. Find students enrolled in Python Programming

```
255 •  SELECT s.StudentID, s.Name
256    FROM Students s
257    WHERE s.StudentID IN (
258        SELECT e.StudentID
259        FROM Enrollments e
260        JOIN Courses c
261            ON e.CourseID = c.CourseID
262        WHERE c.CourseName = 'Python Programming'
263    );
264
```

Result Grid | Filter Rows: | Edit: | Export/Import:

StudentID	Name
101	Arjun
105	Kiran
108	Ananya
113	Nikhil
102	Priya
NULL	NULL